

A Blockchain-Based Distributed and Intelligent Clustering-Enabled Authentication Protocol for UAV Swarms

Raja Karmakar , Member, IEEE, Georges Kaddoum , Senior Member, IEEE, and Ouassima Akhrif , Senior Member, IEEE

Abstract—Unmanned aerial vehicles (UAVs) are operated remotely without the presence of a unified system of identity authentication, and wireless communications in untrusted environments can cause the loss of valuable data carried by UAVs. Traditional UAV authentication mechanisms are centralized approaches, which suffer from a single point of failure problem and may incur high complexity computations. Therefore, it is crucial to establish a distributed authentication mechanism between the ground station controller (GSC) and a UAV. Moreover, in case of UAV swarms, the high mobility of the UAVs affects the stability of UAV communications, which leads to the degradation of the UAV authentication performance. Addressing these challenges, we design a blockchain-based distributed authentication mechanism, known as *SwarmAuth*, for UAV swarms, where the GSC and UAVs follow a mutual authentication approach using physical unclonable functions (PUFs), and the K-means clustering-based intelligent approach is used to dynamically create location-based clusters. The blockchain helps store UAVs' authentication information in an immutable storage and the associated smart contracts provide a convenient access control model. The security analysis of *SwarmAuth* is carried out through both formal and informal proofs considering general attacks. Experimental evaluation shows that *SwarmAuth* can assure trustworthy communications and improve the network performance.

Index Terms—UAV swarms, blockchain, mutual authentication, physical unclonable functions, K-means clustering.

I. INTRODUCTION

UNMANNED aerial vehicle (UAV) technology is recognized as one of the promising aircraft technologies in recent years, with their various capabilities, such as sensing, processing, and delivery of information. Due to their deployment

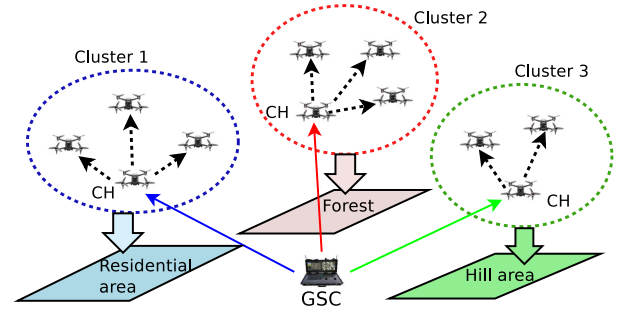


Fig. 1. Illustration of UAV swarm.

flexibility, high-mobility, ability to hover, and low maintenance cost, UAVs can be used in limited access and reachability location-based applications, such as search and rescue, smart agriculture, remote sensing, surveillance, package delivery, and extending wireless network coverage [1]. To this end, swarms of UAVs are designed with advanced sensors in order to facilitate the aforementioned operations with higher levels of accuracy and automation [2]. In UAV swarms, the ground station controller (GSC) communicates with a group of UAVs belonging to a UAV cluster, which changes with the position of the UAVs. Specifically, the GSC communicates with the cluster head (CH), which is responsible for data gathering and transfer between two clusters, and the CH also transmits data between a cluster and the GSC. In a cluster, apart from the CH, the UAVs are known as cluster members (CMs). Fig. 1 shows an example of UAV swarm consisting of three UAV clusters which provide services to three different regions.

Although UAV technologies and applications have been going through rapid development, there are several challenges that hinder their large scale deployment [3]. For instance, in case of UAV-enabled mobile edge computing (MEC)-based services, if UAVs are brought closer to the end users, the consumers will get better services, leading to quality of service (QoS) improvement of UAV applications. However, such deployment results in increased vulnerabilities and high threats that can disrupt the UAV communication. In fact, the messages communicated over wireless channels can be seized by malicious entities, which makes them prone to various attacks, such as man-in-the-middle, node tampering, and replay attacks. Attackers may exploit UAV devices to acquire sensitive information,

Manuscript received 10 May 2023; revised 12 September 2023; accepted 14 September 2023. Date of publication 26 September 2023; date of current version 4 April 2024. This work was supported in part by the Innovation for Defence Excellence and Security (IDEaS) program through the Department of National Defence (DND). Recommended for acceptance by D. Nguyen. (Corresponding author: Raja Karmakar.)

Raja Karmakar is with the Department of Computer Science and Engineering, Heritage Institute of Technology, Kolkata, West Bengal 700107, India (e-mail: rkarmakar.tict@gmail.com).

Georges Kaddoum is with the Department of Electrical Engineering, ETS, University of Quebec, Montreal, QC H3C 3J7, Canada, and also with Cyber Security Systems, and with Applied AI Research Center, Lebanese American University, Beirut 1102-2801, Lebanon (e-mail: georges.kaddoum@etsmtl.ca).

Ouassima Akhrif is with the Department of Electrical Engineering, ETS, University of Quebec, Montreal, QC H3C 3J7, Canada (e-mail: ouassima.akhrif@etsmtl.ca).

Digital Object Identifier 10.1109/TMC.2023.3319544

corrupt the data, cause malicious interference, and distort normal UAV functionalities [4]. Such attacks can drastically affect the output of an operation, which can occur in non-commercial and commercial sectors. For instance, terrorists can use UAVs for malicious purposes, such as impersonating a malicious UAV as a legitimate one. If the legitimate UAV acts as an access point, the attackers gain the access of the access point. Consequently, the malicious UAV injects malware into connected devices through the redirection and interception of users' data traffic or through phishing. As a result, terrorists can acquire and corrupt sensitive data, such as passwords, and distort the functionalities of legitimate UAVs.

Moreover, in the context of UAV swarms, a large number of UAV communications needs to be handled, which also increases the number of security threats.

One of the primary security demands for UAV deployments is designing proper authentication mechanisms which can ensure the participation of only legitimate devices in the data communication [5]. Before initiating a secure and trusted communication session, the first security aspect that needs to be fulfilled is node authentication. In UAV communications, due to highly dynamic nature of the UAVs, the devices are required to be authenticated frequently to ensure that UAVs' normal operations are not affected and the UAV application related information and resources cannot be accessed by a malicious adversary [6]. Moreover, due to the open air deployment of UAVs, they are prone to node capturing attacks, which expose secret keys. Thus, secret information can be obtained by the adversary from the GSC and legitimate UAVs. In addition, since UAVs have limited computation capabilities and memory constraints, it is a challenge to properly execute cryptography algorithms and store the secret keys [7]. Finally, in UAV swarms, the different UAVs should follow the same authentication mechanism, such that they can avoid communication interoperability issues between UAVs of different clusters. Otherwise, the challenges of ensuring authentication security will be compounded during message integration [8].

Considering the aforementioned security issues in UAV communications, a lightweight and scalable UAV authentication mechanism that can achieve a mutual authentication between a UAV and the GSC in a UAV swarm is needed. To resist node capturing attacks, physical unclonable functions (PUFs), which support unique and unclonable identities to devices, are very promising [9]. PUFs use a challenge-response pair, which is unique for every devices, i.e., two different devices with the same manufacturing configuration are expected to provide different responses for the same challenge. The PUF is defined as

$$R = PUF(C), \quad (1)$$

where C and R are binary strings denoting the challenge and response, respectively. Moreover, in order to support the authentication services in a distributed way, blockchain can be exploited. The blockchain is a distributed, immutable, and shared ledger that can enable different devices to approve transactions without involving a central authority [10]. Due to the traceability and unalterable properties of the blockchain, it is resistant to

several attacks [11]. Since, the dimension and mobility of a UAV swarm vary frequently, blockchain can be used to securely store authentication related information in a distributed way for a large number of UAVs, and thus blockchain can help increase the scalability of an authentication approach and make it lightweight to be deployed at resource-constrained UAVs.

A. Related Works and Motivation

1) *Non-Blockchain-Based UAV Authentication:* In [12], a PUF-based authentication protocol is presented for secure communications between base stations and UAVs in UAV swarms, where the K-Means clustering is used to form clusters of UAVs based on their locations. Using PUFs, the work in [6] also designs an authentication mechanism for UAV swarms, which supports spanning tree-based traversal for multi-hop communications. To leverage secure PUFs and programmable high-speed packet-processing data planes, a UAV authentication system is developed in [13]. Wang et al. [14] specifically design a CH safeguarding mechanism for UAV swarms, which utilizes edge intelligence and a situation-aware authentication approach. However, UAV authentication is not addressed in that work. The authors in [15] design UAV-GSC and UAV-UAV authentication schemes using PUFs. The work in [16] addresses secure UAV-UAV communications enabling message integrity, confidentiality, and authenticity. Addressing the tradeoff between lightweight features and security, in [17], a key agreement technique is proposed for internet of drones (IoD). In [18], a decentralized UAV authentication mechanism is designed to overcome the single-point failure in a cluster, caused by the wrong estimations. Furthermore, to reduce the computational cost of the authentication approach and further improve the authentication accuracy of the scheme, a situational-aware authentication mechanism is designed at each UAV. However, the work in [18] does not consider PUFs, blockchain, mutual authentication, and dynamic cluster formation in UAV swarms.

In the case of a non-blockchain-based UAV authentication, no distributed platform is available to store UAV registration and authentication related data such that the data can be shared in a tamper-proof way and can be traceable by the GSC and multiple UAVs, which are authorized to access the data. Moreover, in a non-blockchain-based approach, since authentication related data cannot be securely shared among the GSC and UAVs using a distributed architecture, the number of message transmissions between the GSC and UAV will be higher than that of a blockchain-based approach. As a result, the probability of intercepting messages by attackers will increase.

2) *Blockchain-Based UAV Authentication:* In order to support security in IoT data collection, the work in [19] constructs a proof-of-stake (PoS) consensus-based blockchain among UAVs. In that work, the IoT communication and UAV deployment are optimized for maximizing the blockchain throughput. Considering security and privacy concerns for centralized authentication services in 5G-enabled UAVs, a blockchain-based authentication mechanism is proposed in [20], which particularly addresses the cross domain authentication for drones operating

in different domains. For secure data collection in wireless sensor networks (WSNs), a blockchain-assisted data collection framework is designed in [21] for UAV-enabled WSNs. In addition, an identity authentication mechanism is also developed for UAVs to secure the data transmission. Tan et al. [2] propose a blockchain-enabled distributed authentication solution for industrial UAVs. Here, the proposed mechanism designs smart contracts that help perform secure operations for UAVs to update or acquire the corresponding information. The authors in [22] propose a machine learning-based UAV-base stations deployment mechanism based on computational power, energy, criticality of the network scenario, and nature of data. In addition, to address security issues in untrusted wireless connectivity for UAV-base stations, a blockchain-based information-sharing scheme is proposed. To develop a secure information sharing mechanism for UAV-assisted disaster rescue, Wang et al. [23] propose a blockchain-based architecture to protect data sharing in UAV communications, where vehicular fog computing is introduced to offload UAVs' tasks. However, the aforementioned works do not consider the authentication of group of UAVs in UAV swarms.

3) *Authentication in UAV Swarms*: The work in [24] discusses the application of blockchain for UAV security in industries, along with the evaluation of Hyperledger Fabric-based blockchain technology to UAV networks. The authors in [25] investigate security upgrades supported by blockchain and the improvement of UAV swarm authentication utilizing blockchain-enabled security services. Bansal et al. [26] propose a PUF-based authentication and attestation approach for UAV swarms. Here, the scheme uses an optimal trajectory, which can help establish the trust in the communication. The work in [27] designs a blockchain-assisted trustworthy group communication scheme for UAV networks, where the authors use blockchain to facilitate key distributions and record communication activities. However, this work does not consider PUFs-based security and location-based clustering of UAVs. For multi-UAV coordination and swarm optimization, the authors in [28] propose a model that forms a Wireless Mesh Network (WMN) from the input of the UAVs' locations. In that work, Blowfish and Advanced Encryption Standard (AES) are used to overcome security attacks.

4) *Motivation*: Based on the aforementioned related works, it is noted that there is a lack of research in designing decentralized authentication for UAV groups in UAV swarms. In that direction, location-based UAV cluster formation is important for reducing the message transmission delay, and consequently overall network performance is improved. Moreover, blockchain can help impose a decentralized authentication framework that can be used to store security-related information in a tamper-proof way by multiple UAVs in a swarm. Thus, the blockchain helps guarantee trustworthy communications and data integrity, which are required by several UAV swarm-based applications, such as industrial applications, disaster management, and mission critical applications. Existing works do not investigate the use of blockchain for providing PUF-based authentication services in UAV swarms, where location-based clusters can be formed in an intelligent approach.

B. Our Approach and Contributions

In this paper, by exploiting blockchain, we design a distributed authentication mechanism, *SwarmAuth*, which performs an authentication service for a group of UAVs, i.e., UAV swarms. Each UAV follows UAV registration and authentication with the GSC. In order to create UAV clusters, we use the K-means clustering approach, where we perform location-based clustering of UAVs. The K-means clustering scheme is an unsupervised machine learning algorithm, and thus it is considered an intelligent approach that dynamically forms clusters based on the current position of objects. Since the proposed *SwarmAuth* uses K-means clustering to create UAV clusters, the clustering approach becomes intelligent. The clustering can reduce communication delay and routing overhead, while improving the overall network performance. To the best of our knowledge, this work is the first to jointly consider online learning-based location-aware cluster formation and blockchain-based trusted authentication using PUFs for UAV swarms.

Contributions: Our most notable contributions are summarized as follows.

- Using the PUF, we design a two-way authentication scheme to mutually authenticate the GSC and UAV. Based on the individual authentication mechanism, we design an authentication approach for UAV swarms. The authentication services are provided cluster-wise, where the messages targeted for a set of UAVs in a cluster are aggregated and then transmitted from the GSC to the cluster and vice versa.
- To dynamically generate the UAV cluster, we use a K-means clustering-based online learning mechanism, where a cluster is formed dynamically considering the location of the UAVs.
- We introduce a blockchain-based authentication service for UAV swarms. The information required for the authentication is stored in the blockchain, and we design smart contracts to impose access control mechanisms on the secure information.
- A formal security analysis is provided using the Mao and Boyd logic [29] and Scyther tool [30]. Considering several mainstream attacks, we also present an informal security proof.
- We implement the proposed blockchain in Truffle, which is a popular Ethereum blockchain development and testing framework. The blockchain peer nodes are hosted on the cloud server. Evaluation results show the access controls of the users and the time cost of the smart contract APIs designed in the blockchain.
- We implement *SwarmAuth* in NS-3.36 [31], where the practical impact of *SwarmAuth* is assessed using several metrics, such as the average network throughput, delay, and communication and computation costs.

C. Organization of This Paper

The organization of this paper is as follows. Section II presents the design philosophy of *SwarmAuth*. Section III discusses the authentication and group message transmission procedure. Section IV describes the proposed blockchain-based authentication

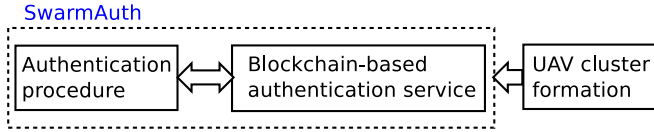


Fig. 2. SwarmAuth design framework.

service. Section V discusses the UAV cluster formation mechanism in SwarmAuth. In Section VI, a thorough security analysis of SwarmAuth is presented. Then, in Section VII, we present a comparative performance analysis of SwarmAuth with related mechanisms and evaluate the performance of the blockchain. Finally, Section VIII concludes the paper.

II. SWARMAUTH: DESIGN PHILOSOPHY

In this section, we present the design details of SwarmAuth, which has three primary objectives as follows:

- 1) Defining an authentication process between the GSC and a UAV,
- 2) Implementing the UAV-GSC authentication process using a blockchain framework, such that a decentralized authentication approach can be achieved to store the authentication related information in the blockchain, and
- 3) Dynamically generating the UAV cluster considering the location of the UAVs, which will take the blockchain-based authentication service for the UAV-GSC authentication.

Accordingly, we define three primary functionalities – (i) authentication procedure, (ii) blockchain-based authentication service, and (iii) UAV cluster formation, as shown in Fig. 2. Once UAV clusters are formed, the UAVs use the blockchain-based UAV-GSC authentication service. Further details about these functionalities are discussed below.

- 1) *Authentication procedure*: This functionality defines the UAV-GSC authentication process and also describes the group message communication approach in a UAV swarm in SwarmAuth. The authentication procedure is done over two phases – (a) registration and (b) mutual authentication. In the registration phase, a UAV registers with the GSC, while in the mutual authentication phase, a UAV and the GSC authenticate each other to start UAV communications.
- 2) *Blockchain-based authentication service*: This functionality defines a blockchain architecture that is used to provide a distributed authentication service based on the aforementioned authentication procedure. Blockchain peer nodes are stored in the cloud server and can be accessed by the GSC and the UAVs.
- 3) *UAV cluster formation*: This functionality describes the mechanism to intelligently generate location-based UAV clusters using the K-means clustering approach, considering the position of each UAV while forming clusters. After generating UAV clusters, the UAV cluster formation functionality takes services from the other two functionalities for UAV authentication using a blockchain platform.

TABLE I
LIST OF PRIMARY NOTATIONS

Notation	Description
TID_v	Temporary ID of V
UID_v	Permanent ID of V
N_v	Random nonce generated by V
$GSID$	ID of G
(C_v, R_v)	PUF challenge-response pair of V
$h()$	Hash function
DIC_v	Dynamic identity code (DIC) of V
SK_v	Session key for secure communication between G and V
$\oplus$	XOR operation
$\parallel$	Concatenation operation
F_k	List of UAVs in k^{th} flow (or cluster)
M_{kj}^R	Message transmitted to j^{th} UAV in the k^{th} flow, during registration
M_{kj}^{Ai}	Message transmitted to j^{th} UAV in the k^{th} flow, during authentication ($1 \leq i \leq 2$)
$E(M_{kj}^R)_{key}$	M_{kj}^R encrypted with $key = (R_{kj} \oplus N_{kj})$
$E(M_{kj}^{Ai})_{key}$	M_{kj}^{Ai} encrypted with $key = (R_{kj} \oplus N_{kj})$ ($1 \leq i \leq 2$)
$M^{R,aggr}$	Aggregated message transmitted in the k^{th} flow, during registration
$M^{Ai,aggr}$	Aggregated message transmitted in the k^{th} flow, during authentication ($1 \leq i \leq 2$)
(x_i, y_i, z_i)	Position of UAV v_i
K	Number of UAV clusters
C	Set of cluster centers

Table I shows the primary notations used in this paper. Next, we present details of the registration and authentication approaches defined in the authentication procedure.

III. AUTHENTICATION PROCEDURE

The proposed registration and authentication approaches are followed by each UAV in a UAV swarm.

A. Registration

Before the deployment, UAV V registers with GSC G . To this end, the UAV generates a dynamic identity code (DIC), which is shared with the GSC. The registration process follows two steps, as described in what follows. Fig. 3 shows different messages used for UAV registration in SwarmAuth. Details of the registration approach are described in what follows (the brief of a step is mentioned inside the parenthesis).

Step-1 (DIC generation at UAV):

- 1) V randomly generates a challenge C_v , then applies the PUF to produce the response R_v as

$$R_v = \text{PUF}(C_v).$$

- 2) A random nonce, N_v , is generated, based on which a temporary ID, TID_v , is generated by V , as

$$TID_v = h(UID_v, N_v).$$

After registration, TID_v is used to uniquely identify a UAV during authentication.

- 3) TID_v is split into $D1$ and $D2$. In order to retrieve N_v at the GSC-end, a parameter Q_v is calculated by encoding N_v as

$$Q_v = N_v \oplus TID_v \oplus D1 \oplus D2.$$

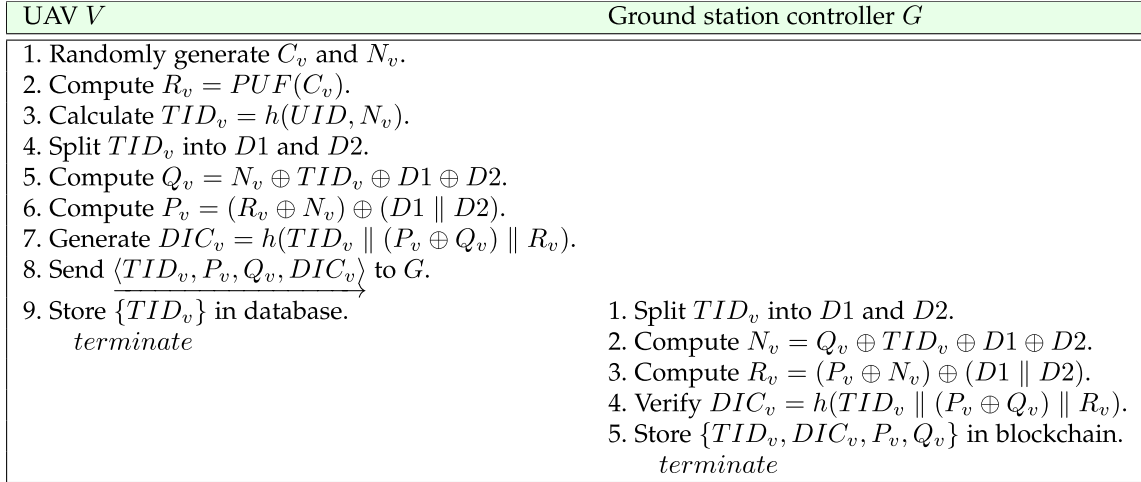


Fig. 3. UAV registration.

- 4) To retrieve R_v at the GSC, a parameter P_v is computed as

$$P_v = (R_v \oplus N_v) \oplus (D1 \parallel D2).$$

- 5) V 's DIC, denoted by DIC_v , is computed as

$$DIC_v = h(TID_v \parallel (P_v \oplus Q_v) \parallel R_v).$$

- 6) V sends TID_v , P_v , Q_v , and DIC_v to the GSC and stores $\{TID_v\}$ in its database. Let M_v^R denote the message sent by V to the GSC, and thus $M_v^R = \{TID_v, DIC_v, P_v, Q_v\}$.

Step-2 (DIC verification at GSC):

- 1) When G receives TID_v from V , G splits TID_v into $D1$ and $D2$, following the approach used by V .
- 2) The GSC decrypts N_v from Q_v as

$$N_v = Q_v \oplus TID_v \oplus D1 \oplus D2.$$

- 3) Based on N_v , R_v is decrypted from P_v as

$$R_v = (P_v \oplus N_v) \oplus (D1 \parallel D2).$$

- 4) Based on TID_v , P_v , and R_v , the value of DIC_v is computed and it is verified with DIC_v sent by V . This verification ensures that the correct information is retrieved by the GSC.
- 5) The GSC stores the tuple $\{TID_v, DIC_v, P_v, Q_v\}$ in the blockchain.

Therefore, at the end of the registration, the blockchain contains the tuple $\langle TID_v, DIC_v, P_v, Q_v \rangle$ for each UAV in a swarm. In general, this tuple is represented as $\langle TID, DIC, P, Q \rangle$. The GSC can add or delete records of the ledger in the blockchain. In order to avoid information loss by a UAV, UAVs can only update TID , DIC , P , and Q . Each UAV can only update its own information, while GSC as UAVs' registrar has the permission to add or delete records of the ledger. TID_v is divided into $D1$ and $D2$, so if an adversary guesses TID_v , they cannot use it to calculate Q_v , P_v , and DIC_v , which are computed by the UAV through $D1$ and $D2$. Consequently, these two parts are also employed by the GSC to decrypt N_v and R_v . As a result,

decomposing TID_v increases the security level in the UAV authentication process.

B. Mutual Authentication

After a mutual authentication establishment between G and V , a secure session key is generated between them, which is used for subsequent secure communication between these parties. For each new authentication, V generates a new C_v and N_v . Accordingly, P_v and Q_v are updated. This approach ensures the freshness of an authentication request. Both UAVs and the GSC do not store N_v and R_v in their databases. After the successful registration of UAVs or the completion of an authenticated session, UAVs compute $\langle TID, DIC, P, Q \rangle$ for the next authentication process and update the tuple accordingly in the blockchain. This approach is taken to avoid the time it takes to update the data in the next authentication process. Fig. 4 shows different messages used for the mutual authentication in SwarmAuth. Details of the authentication approach are described in what follows.

Step-1 (Authentication initiation at UAV):

- 1) When V wants to establish a communication session with the GSC, it randomly generates N_v and C_v and then applies the PUF to produce R_v from C_v .
- 2) V recomputes TID_v , P_v , Q_v , and DIC_v and updates them in the blockchain.
- 3) In order to authenticate V at the GSC, TID_v is sent to G . At this step, let M_v^{A1} denote the message sent by V to G , where $M_v^{A1} = \{TID_v\}$.

Step-2 (UAV verification and session key generation at GSC):

- 1) After getting $\{TID_v\}$ from the UAV, G splits TID_v into $D1$ and $D2$ by using the mechanism followed at registration time. From the blockchain, the GSC finds P_v and Q_v corresponding to TID_v sent by the UAV.
- 2) Based on TID_v , P_v , and Q_v , the GSC computes N_v and R_v .
- 3) The GSC calculates $DIC_v = h(TID_v \parallel (P_v \oplus Q_v) \parallel R_v)$. If it matches the DIC_v corresponding to TID_v

UAV V	Ground station controller G
1. Randomly generate N_v and C_v. 2. Calculate $R_v = PUF(C_v)$. 3. Compute $TID_v = h(UID, N_v)$. 4. Calculate $Q_v = N_v \oplus TID_v \oplus D1 \oplus D2$. 5. Calculate $P_v = (R_v \oplus N_v) \oplus (D1 \parallel D2)$. 6. Compute $DIC_v = h(TID_v \parallel (P_v \oplus Q_v) \parallel R_v)$. 7. Update TID_v, P_v, Q_v, and DIC_v in blockchain. 8. Send $\langle TID_v \rangle$ to G.	1. Split TID_v into $D1$ and $D2$. 2. Find P_v and Q_v from blockchain, for TID_v. 3. Compute $N_v = Q_v \oplus TID_v \oplus D1 \oplus D2$. 4. Compute $R_v = (P_v \oplus N_v) \oplus (D1 \parallel D2)$. 5. Verify $DIC_v = h(TID_v \parallel (P_v \oplus Q_v) \parallel R_v)$. 6. Generate $GVC = h(TID_v \parallel GSID \parallel R_v \parallel N_v)$. 7. Compute $SK_v = h(TID_v \parallel (R_v \oplus N_v) \parallel DIC_v)$. 8. Generate $SKC_v = h(SK_v \parallel GVC \parallel R_v \parallel N_v)$. 9. Send $\langle GVC, SKC_v \rangle$ to V.
9. Verify $GVC = h(TID_v \parallel GSID \parallel R_v \parallel N_v)$. 10. Calculate $SK_v = h(TID_v \parallel (R_v \oplus N_v) \parallel DIC_v)$. 11. Verify $SKC_v = h(SK_v \parallel GVC \parallel R_v \parallel N_v)$. <i>terminate</i>	<i>terminate</i>

Fig. 4. UAV authentication.

stored in the blockchain, V is successfully verified as a registered UAV.

- 4) The GSC generates a GSC verification code (GVC) as

$$GVC = h(TID_v \parallel GSID \parallel R_v \parallel N_v),$$

where $GSID$ is the unique identifier (ID) of the GSC. The GVC is used to verify the GSC at the UAV end.

- 5) For further communication with V , the session key, denoted by SK_v , is computed as

$$SK_v = h(TID_v \parallel (R_v \oplus N_v) \parallel DIC_v).$$

- 6) To verify the session key at V , a session key code (SKC) is generated as

$$SKC_v = h(SK_v \parallel GVC \parallel R_v \parallel N_v).$$

- 7) The GSC sends GVC and SKC_v to V . At this step, let M_v^{A2} denote the message sent by G to V , where $M_v^{A2} = \{GVC, SKC_v\}$.

Step-3 (GSC verification at UAV):

- 1) V computes $GVC = h(TID_v \parallel GSID \parallel R_v \parallel N_v)$ and verifies it against the GVC sent by the GSC.
- 2) After the successful verification of the GSC, the UAV calculates $SK_v = h(TID_v \parallel (R_v \oplus N_v) \parallel DIC_v)$. Then, SKC_v is computed, and if it matches the SKC_v sent by V , the session key SK_v is verified.

In general, M_v^R , M_v^{A1} , and M_v^{A2} are represented as M^R , M^{A1} , and M^{A2} , respectively. Next, we discuss the proposed group message transmission procedure for a UAV swarm.

C. Group Message Transmission

Based on the individual registration and authentication mechanisms discussed in Sections III-A and III-B, SwarmAuth performs an authentication for a cluster of UAVs. In order to communicate with the UAVs in a cluster, the GSC creates a flow path connecting the GSC with the CH in a cluster. The communication between the GSC and UAVs are carried out in a hop-by-hop approach. The GSC provides the services cluster-wise, and for that purpose, messages for a cluster are aggregated and then transmitted to the cluster. In this context, the GSC G first transmits a message to V_i , which is the CH of the i^{th} cluster. Then, V_i forwards the message to other UAVs in the cluster. For UAV registration and authentication, since messages are communicated between the GSC and the UAVs, the mutual authentication is performed between the GSC and a UAV. In this context, the CH only performs forwarding of messages to the UAVs in a cluster. Specifically, clusters determine the number of flow paths for the UAV communication. Thus, for K clusters, K flows (or paths) are formed, which are denoted by $\{F_1, F_2, \dots, F_K\}$, where F_k ($1 \leq k \leq K$) denotes the set of UAVs in the k^{th} flow. Let n^k be the number of UAVs in F_k , and thus $n^k = |F_k|$. At the time of registration, let M_{kj}^R be the message transmitted to j^{th} UAV in flow k . At the authentication time, let M_{kj}^{A1} and M_{kj}^{A2} be the messages transmitted to the j^{th} UAV in flow k . Fig. 5 shows the group message transmission in a cluster.

The location of the UAVs is not considered in the proposed authentication mechanism, and therefore, the cluster formation approach does not impact the authentication process. However, as the number of clusters increases, the number of aggregated

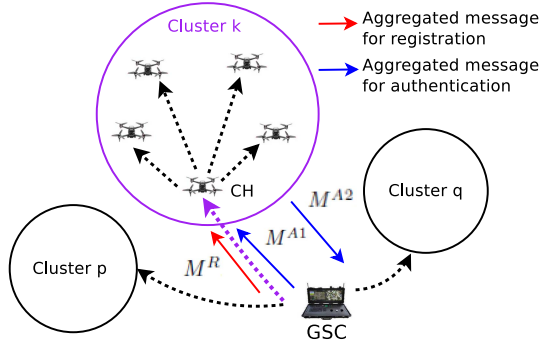


Fig. 5. Group message transmission in a cluster.

messages used for authentication services are increased. This is because authentication services for the UAVs are provided cluster-wise by aggregating the messages destined for a cluster.

1) *Message Encryption*: The PUF response and nonce value are changed in each authentication initiation. Thus, to ensure message authentication and freshness in an authentication approach, each M_{kj}^R is encrypted with $key = (R_{kj} \oplus N_{kj})$, where R_{kj} and N_{kj} are the PUF response and random nonce of the j^{th} UAV in the k^{th} flow, respectively. Let $E(M_{kj}^R)_{key}$ represent the message M_{kj}^R encrypted by key . For the encryption purpose, the AES is used.

2) *Flow Paths in Registration*: To create the aggregated message for flow k , a message L_{kj}^R is constructed for UAV j by appending DIC_{kj} before the encrypted value of M_{kj}^R . The purpose of this aggregation is to enable the UAV to decrypt messages intended for it by finding its DIC_k in the flow and decrypting the message using $key = (R_{kj} \oplus N_{kj})$. Even if an adversary manages to guess DIC_{kj} for the j^{th} UAV in flow k , they will not be able to decrypt the corresponding message M_{kj}^R because it is encrypted with $key = (R_{kj} \oplus N_{kj})$. Therefore, the adversary would need to guess R_{kj} , N_{kj} , and the key to decrypt the message. In general, for the k^{th} flow, we represent L_{kj}^R as L_k^R . Thus, we have

$$L_{kj}^R = DIC_{kj} \parallel E(M_{kj}^R)_{key}, \quad (2)$$

where $1 \leq j \leq n^k$ and DIC_{kj} is the DIC of the j^{th} UAV in the k^{th} flow. Based on DIC_{kj} , L_{kj}^R helps the j^{th} UAV identify its message M_{kj}^R . Then, considering all the UAVs in the k^{th} flow, the aggregated message is formed. Let $M_k^{R,aggr}$ represent the aggregated message, defined as

$$M_k^{R,aggr} = L_{k1}^R \parallel L_{k2}^R \parallel \dots \parallel L_{kn^k}^R. \quad (3)$$

3) *Flow Paths in Authentication*: For the authentication purpose, let L_{kj}^{Al} represent the concatenation of DIC_{kj} and M_{kj}^{Al} , and $M_k^{Al,aggr}$ be the aggregated authentication messages, where $l \in \{1, 2\}$. Now, based on (2) and (3), L_{kj}^{Al} and $M_k^{Al,aggr}$ are defined for the authentication. Thus, L_{kj}^{Al} can be defined as

$$L_{kj}^{Al} = DIC_{kj} \parallel E(M_{kj}^{Al})_{key}, \quad l \in \{1, 2\}. \quad (4)$$

Therefore, the aggregated message $M_k^{Al,aggr}$ can be represented as

$$M_k^{Al,aggr} = L_{k1}^{Al} \parallel L_{k2}^{Al} \parallel \dots \parallel L_{kn^k}^{Al}, \quad l \in \{1, 2\}. \quad (5)$$

D. Intended Message Identification in Cluster

During registration (or authentication), when $M_k^{R,aggr}$ (or $M_k^{Al,aggr}$) is transmitted to a cluster, each UAV first checks DIC_k in L_k^R (or L_k^{Al}). If a UAV finds its DIC_k in the flow, the message followed by DIC_k is intended for the UAV, and thus it decrypts the message using key .

E. Difference Between Our Approach and the Existing PUF-Based UAV Authentication Mechanism in [15]

The authors in [15] designed UAV-GSC and UAV-UAV authentication schemes using PUFs. However, our work proposes a UAV-GSC authentication mechanism for UAV swarms using PUFs. Moreover, the work in [15] does not address the authentication for UAV swarms. In addition, blockchain is not used in [15], and thus the authentication mechanism proposed in [15] is not a distributed approach, where the GSC stores several authentication related information, such as the challenge-response pair and the temporary ID of UAVs, in its database. Furthermore, the UAV stores the challenge value, the temporary ID of the UAV, and the ID of the GSC in the UAV's database. However, in our approach, the GSC does not store any authentication related data in its database, and the UAV stores only its temporary ID in its database. At the end of the registration, the blockchain contains the information for each UAV in a swarm.

In the next section, we elaborate on the blockchain architecture for the authentication service proposed in SwarmAuth.

IV. BLOCKCHAIN-BASED AUTHENTICATION SERVICE

In this section, we describe the proposed blockchain architecture and the working flow of the authentication service.

A. Architecture

Since blockchain eases the process of securely storing data and recording transactions, we apply it for the following two purposes.

- 1) We use blockchain as a distributed storage to store the parameters required for the registration and authentication in a tamper-proof way. We call these parameters Security Information (SI). As discussed in Section III, the blockchain stores $\langle TID, DIC, P, Q \rangle$, and therefore SI includes $\langle TID, DIC, P, Q \rangle$.
- 2) Using smart contracts, we restrict the operations on security parameters, such that only authorized users can update those parameters.

The proposed system architecture of the blockchain-based authentication service is shown in Fig. 6. Each of the blockchain peer nodes stores a copy of the ledger that holds the security information. Thus, a 'record' of the ledger contains the security information. Any update in this information results in changes in the ledger, leading to a new block in the blockchain. Thus,

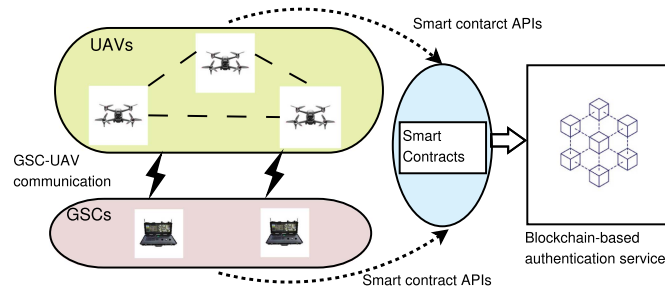


Fig. 6. System model for SwarmAuth.

every change in the blockchain is incrementally recorded by the blockchain, and consequently the UAV authentication service becomes unforgeable and traceable. To automatically execute operations (such as addition, update or deletion) on the security information, we apply smart contracts which are installed on blockchain peer nodes. The GSC and UAVs can call smart contract APIs and trigger executions of smart contracts. Before the authentication, a UAV needs to be registered in the GSC, and UAVs have the permission to only update their own information. In our approach, we introduce a validation step, where blockchain peer nodes validate changes of the ledger before finally being recorded into the blockchain.

Next, we discuss the key components of the proposed blockchain architecture. Based on these components, the working flow of SwarmAuth is defined.

B. Key Components

In SwarmAuth, the key components of the blockchain include blockchain nodes, ledger, and smart contracts.

1) *Blockchain Nodes*: In the proposed blockchain, we introduce two types of nodes – (i) endorsement nodes and (ii) validation nodes. All blockchain peer nodes act as endorsement nodes, which are responsible for receiving transaction proposals from the GSC or UAVs, checking the permission to update the ledger, executing the appropriate smart contracts and returning the results as proposal responses to validation nodes. These verify and validate the endorsements. If they are verified, validation nodes create new blocks and add them into the blockchain. Thus, validation nodes have the responsibility for validating each new block before it is inserted into the blockchain. Validation nodes are randomly selected from blockchain peer nodes. Each of the endorsement and validation nodes holds a copy of the updated ledger, and the smart contracts are installed on endorsement/validation nodes to execute modifications in the blockchain. Validation nodes ensure that copies of the ledger hosted by blockchain peer nodes remain the same.

2) *Ledger and Smart Contracts*: To realize the registration and authentication service, we designed a ledger known as Security Information Ledger (SIL), which stores $\langle TID, DIC, P, Q \rangle$. For SIL, a smart contract called Security Information Contract (SIC) is designed, which contains several APIs for the GSC or UAVs to invoke at the time of the UAV registration or authentication process. Details of the smart contract are discussed below.

Smart Contract Operations: In the smart contract, the APIs perform the following operations: (i) *saveSI*, (ii) *deleteSI*, (iii) *updateSI*, (iv) *queryPQ*, and (v) *queryDIC*. The details of these operations are defined below.

- *saveSI* is used to store a new SI into SIL, and it can only be invoked by the GSC. *deleteSI* is the opposite function of *saveSI*, used to delete an SI from SIL. It can only be invoked by the GSC. The SI of a UAV is deleted when the UAV loses connection with the GSC or is identified as malicious. We define a permission mechanism in the SIC to ensure that the *saveSI* and *deleteSI* APIs are accessed by the GSC only. If any UAV requests access to these APIs, the request is refused by the SIC.
- To update the SI of a UAV, *updateSI* is used by the UAV, where it can only update its own *TID*, *DIC*, *P*, and *Q*, corresponding to the UAV's old *TID*.
- We also define two query APIs, *queryPQ* and *queryDIC*, which are used to find *P*, *Q*, and *DIC* corresponding to the *TID* of a UAV. Specifically, for a given *TID*, *queryPQ* returns *P* and *Q*, and *queryDIC* returns *DIC*. Therefore, these two query APIs are used by the GSC to ask for *P*, *Q*, or *DIC* for a *TID* supplied to the SIC. Since these values are modified for a UAV before the beginning of the authentication approach, the GSC receives fresh values of these parameters for each authentication. Furthermore, a UAV does not have permission to get *P*, *Q*, and *DIC* for another UAV.

Determination of Freshness of SI: After the authentication process of UAV *V* is completed, the GSC stores N_v , which helps check the freshness of an authentication initiation in the future. This is because each authentication procedure should generate a new N_v . The freshness of N_v is reflected in *P*, *Q*, and *DIC*. Therefore, if a UAV fails to update these values before the authentication time, the GSC will refuse its authentication initiation request.

3) *Smart Contract Applications*: The blockchain platform provides a Software Development Kit (SDK) that uses a set of functionalities to define a blockchain. We use the SDK to install smart contract applications on the GSC and UAVs. The applications enable them to create transaction proposals, which can be used to update the SI in the SIL. Then, to submit the proposals, appropriate smart contract APIs are invoked by the smart contract applications. All applications are executed in a listening mode, such that the submission of any transaction proposal can be captured and the corresponding API is invoked.

C. Working Flow of Updating SIL

The SIL update procedure is shown in Fig. 7. For instance, we assume that *V* wants to update its SI in order to prepare it for the authentication initiation. First, it is required to send a transaction proposal by *V*, and for that purpose, *V* calls *updateSI* through the smart contract application installed on *V* and sends the proposal to endorsement nodes situated near it. The nodes check whether *V* has the permission to update the corresponding SI. If the update request is validated, the endorsement nodes execute SIC and return the results to validation nodes as a transaction

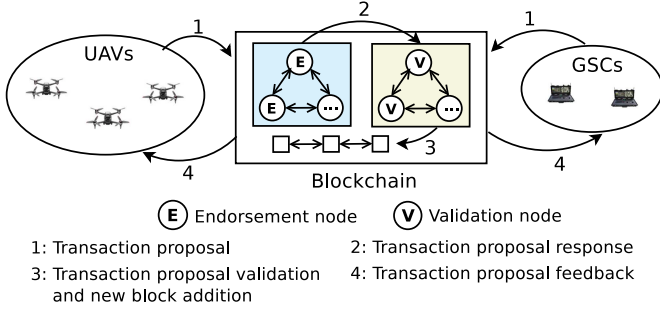


Fig. 7. Working flow of SwarmAuth.

proposal response. The endorsement nodes forward the results to their nearby validation nodes. After receiving enough responses, these nodes validate the proposal response, and if it is verified, a new block is inserted into the blockchain. As a result, the SIL of V is updated according to the executed transaction. Therefore, after this step, the SIL will reflect the updated value, which can be accessed by the GSC. Lastly, a feedback is sent to V informing it about the status of the transaction proposal result, i.e., success or failure, where success indicates that the proposal is executed successfully. When the GSC wants the SI of V , it accesses the blockchain and runs a query proposal (*queryPQ* or *queryDIC*), which is processed by the endorsement and validation nodes following the aforementioned steps.

Next, we discuss a K-means clustering-based UAV cluster formation mechanism used in SwarmAuth.

V. UAV CLUSTER FORMATION

Let the set of UAVs be modeled as $U = \{v_1, v_2, \dots, v_n\}$. By sending messages, UAVs exchange information including identification numbers of the cluster and UAV, role of a UAV (CH or CM), position of a UAV, distance between a UAV and the corresponding CH, and distance between a UAV and the GSC.

A. Determination of the Number of Clusters

To reduce the cluster formation overhead and utilize the channel bandwidth efficiently, it is required to determine the optimal number of clusters for the clustering center selection. In a cluster, let the throughput of a CM be T_M , which can be defined as [32]

$$T_M = \Theta(B_1 / \sqrt{n/K}). \quad (6)$$

Similarly, the throughput of the CH is defined as [32]

$$T_H = \Theta(B_2 / \sqrt{K}). \quad (7)$$

Here, n is the total number of UAVs in the network, and K denotes the number of clusters. B_1 represents the bandwidth of intra-cluster communications, and B_2 is the bandwidth of inter-cluster communications. In order to maintain the throughput balance between the aforementioned two cluster communications, the following relationship needs to be

maintained [33].

$$\frac{K-1}{K} T_M \leq T_H \quad (8)$$

When $\frac{K-1}{K} T_M$ reaches the maximum value, and n is large enough, the value of K is obtained as

$$K = \frac{B_2}{B_1} \sqrt{n}. \quad (9)$$

B. Clustering Center Selection

Using the uniform distribution, the initial clustering center, denoted by c_1 , is randomly chosen from the set U . Let $D(v_i)$ denote the minimum value of the euclidean distance between the currently chosen clustering center and v_i , where $1 \leq i \leq n$. Let $P(v_i)$ be the probability of choosing v_i as the next cluster center. We compute $P(v_i)$ as follows.

$$P(v_i) = \frac{D(v_i)^2}{\sum_{v_i \in U} D(v_i)^2} \quad (10)$$

The UAV which has the highest $P(v_i)$ is selected as the next cluster center. This process is repeated until K centers are selected, and let the set of clustering centers be expressed as $C = \{c_1, c_2, \dots, c_K\}$. We use the K centers for the K-means clustering algorithm.

C. Cluster Formation

Based on the closest cluster center, the set U is divided into K clusters using the K-means clustering algorithm [34]. To this end, after generating C , we compute the distance between each UAV and the cluster centers. Then, we compute the minimum distance $d_{cv_p}^{min}$, which is expressed as

$$d_{cv_p}^{min} = \min(d_{c_1 v_p}, d_{c_2 v_p}, \dots, d_{c_K v_p}). \quad (11)$$

Based on (11), we assign each UAV to its nearest cluster center. For instance, if $d_{cv_p}^{min} = d_{c_i v_p}$, UAV v_p is clustered into the i^{th} cluster. This process is repeated until K clusters are formed with all the UAVs. Let (x_p, y_p, z_p) be the position of v_p , where x_p , y_p , and z_p are X, Y, and Z coordinate values, respectively. Since UAVs can have high mobility, the clustering center values are updated as

$$C = \frac{1}{J_i} \left(\sum_{p=1}^{J_i} x_p, \sum_{p=1}^{J_i} y_p, \sum_{p=1}^{J_i} z_p \right), \quad (12)$$

where J_i is the number of UAVs in the i^{th} cluster. Therefore, based on the positions of the UAVs, C is changed, and consequently, the clusters of the UAVs are updated dynamically. After the formation of clusters, when UAV v_p travels further than a distance of $d_{c_i v_p}$, the distance between UAV v_p and the corresponding cluster center is recomputed. If the calculated distance becomes greater than $d_{c_i v_p}$, C is updated following (12). This is because the distance $d_{c_i v_p}$ exceeds its last distance from the associated cluster center, leading to a high probability of leaving the cluster by the UAV.

VI. SECURITY ANALYSIS

The formal security proof of SwarmAuth is analyzed using the broadly-accepted Mao and Boyd logic [29] and Scyther tool [30]. In addition, we also provide an informal proof, which uses the old-fashioned cryptanalysis and guarantees versatility and security [35], [36].

A. Formal Security Proof Using Mao and Boyd Logic

For the formal security proof using Mao and Boyd logic, we apply the nonce-verification, authentication, confidentiality, good-key, super-principal, and fresh inference rules [29]. To this end, we prove the statements “ V believes N_v is a good secret key between V and G ” and “ G believes N_v is a secret key between V and G ”.

Claim 1: V believes N_v is a secret key between V and G .

Proof: Since R_v is the output of the PUF, and G receives R_v in encrypted format, we can have the statement “ V believes R_v is a good secret between V and G ” (13). Moreover, each authentication process is initiated with a new nonce N_v , which is only shared with G in encrypted format. Thus, “ V believes none other than G have access to N_v ” (14). V uses R_v to encrypt N_v (15). The *confidentiality rule* is applied to (13), (14), and (15), which yields “ V believes none other than V and G have access to N_v ” (16).

$$V \equiv V \xrightarrow{R_v} G. \quad (13)$$

$$V \equiv \{G\}^c \triangleleft N_v. \quad (14)$$

$$V \xrightarrow{R_v} N_v. \quad (15)$$

$$V \equiv \{V, G\}^c \triangleleft N_v. \quad (16)$$

At each authentication initiation, since a new N_v is generated, “ V believes N_v is fresh” (17). Applying the *good-key rule* to (16) and (17), proves that “ V believes N_v is a secret key between V and G ” (18).

$$V \equiv \#(N_v). \quad (17)$$

$$V \equiv V \xrightarrow{N_v} G. \quad (18)$$

Claim 2: G believes N_v is a secret key between V and G .

Proof: R_v is unique to V and is transmitted to G in encrypted form. Moreover, R_v is computed using $D1$ and $D2$, which are only known to V and G . Thus, “ G believes R_v is a good secret between V and G ” (19). G can decipher N_v using R_v (20). Now, the *authentication rule* is applied to (19) and (20), and consequently the statement, “ G believes V encrypted N_v using R_v ”, is obtained (21). In each authentication process, V creates a new N_v , and thus “ G believes N_v is fresh” (22).

$$G \equiv U \xrightarrow{R_v} G. \quad (19)$$

$$G \xrightarrow{R_v} N_v. \quad (20)$$

$$G \equiv U \xrightarrow{R_v} N_v. \quad (21)$$

$$G \equiv \#(N_v). \quad (22)$$

Now, we apply the *nonce-verification rule* to (21) and (22) and get “ G believes V believes that R_v is a good secret between V and G ” (23). Since N_v is new in each authentication, we have the statement “ G believes V believes that none other than G have access to N_v ” (24). The *confidentiality rule* is applied to statements (21), (23), and (24), and we obtain “ G believes V believes that no one other than V and G have access to N_v ” (25). It is considered that V is legitimate and G believes V is trusted (26).

$$G \equiv V \equiv \xrightarrow{R_v} G. \quad (23)$$

$$G \equiv V \equiv \{G\}^c \triangleleft N_v. \quad (24)$$

$$G \equiv V \equiv \{V, G\}^c \triangleleft N_v. \quad (25)$$

$$G \equiv \text{sup}(V). \quad (26)$$

Using the *super-principal rule* on (25) and (26), we get “ G believes no one other than G and V have access to N_v ” (27). Now, we apply the *good-key rule* to (22) and (27) and obtain “ G believes N_v is a secret key between V and G ” (28).

$$G \equiv \{V, G\}^c \triangleleft N_v. \quad (27)$$

$$G \equiv V \xrightarrow{N_v} G. \quad (28)$$

Hence, adversaries cannot access N_v . Similarly, it can be proved that adversaries cannot access R_v . Therefore, if N_v and R_v cannot be accessed by adversaries, the secrecy of these parameters is maintained in different types of attacks, such as man-in-the-middle, masquerade, and replay attacks.

B. Formal Security Analysis Using Scyther

Scyther [30] is a security analysis simulator that verifies security protocols with different number of sessions. We implement SwarmAuth in Scyther, where we set (i) the search pruning to “Find All Attacks”, (ii) the matching type to “Find All Type Flaws”, (iii) the number of runs to 10, and (iv) the per claim maximum number of patterns to “10”. In our implementation in Scyther, the name of the protocol is SwarmAuth, in which ‘ G ’ and ‘ V ’ are known as ‘roles’. Each of them states 7 claims, as shown in Fig. 8, where it is noted that both R_v and N_v are secret between G and V , which is mentioned in claims 1 and 2 (in both V and G). The general synchronous property and consistency of the protocol are defined by claim 3 (*Nisynch*), which also ensures that a message receiving event is preceded by a message sending event. In claim 4 (*Niagree*), it is stated that a role who is acting as an initiator and performing one-way authentication with another one acting as a responder believes that after the execution of the protocol, the data consistency is achieved with the responder. In claim 5 (*Commit*), the successful execution of the protocol is ensured maintaining the secrets of N_v and R_v . Claim 6 (*Alive*) maintains the liveness of G and V throughout a protocol execution. If it is believed by a role that it can communicate with another one, claim 7 (*Weakagree*) is satisfied.

Scyther results : verify

Claim				Status	Comments
SwarmAuth	V	SwarmAuth,i1	Secret Nv	Ok	No attacks within bounds.
		SwarmAuth,i2	Secret Rv	Ok	No attacks within bounds.
		SwarmAuth,i3	Nisynch	Ok	No attacks within bounds.
		SwarmAuth,i4	Niagree	Ok	No attacks within bounds.
		SwarmAuth,i5	Commit G,Nv,Rv	Ok	No attacks within bounds.
		SwarmAuth,i6	Alive	Ok	No attacks within bounds.
		SwarmAuth,i7	Weakagree	Ok	No attacks within bounds.
G		SwarmAuth,r1	Secret Nv	Ok	No attacks within bounds.
		SwarmAuth,r2	Secret Rv	Ok	No attacks within bounds.
		SwarmAuth,r3	Nisynch	Ok	No attacks within bounds.
		SwarmAuth,r4	Niagree	Ok	No attacks within bounds.
		SwarmAuth,r5	Commit V,Nv,Rv	Ok	No attacks within bounds.
		SwarmAuth,r6	Alive	Ok	No attacks within bounds.
		SwarmAuth,r7	Weakagree	Ok	No attacks within bounds.

Done.

Fig. 8. Result of formal security analysis using Scyther.

C. Informal Security Analysis

Using informal security proof, we analyze the security of SwarmAuth considering different well-known security attacks, as discussed below.

1) *Mutual Authentication*: In each authentication process, new (C_v, R_v) and N_v are created, and they are not stored in G 's or V 's database. Moreover, V transmits these values to G in encrypted format. Therefore, the session key SK_v , generated using R_v and N_v , is unique in each sessions and can be successfully computed only between G and V . Hence, mutual authentication can be achieved between these two parties.

2) *Forward Secrecy*: In a session, if an adversary is able to guess SK_v , the security of the next session will not be hampered. This is because the (C_v, R_v) pair, R_v , and N_v are fresh in each new authentication initiation, and accordingly, SK_v and DIC_v are changed in every session. Moreover, based on R_v and N_v ,

SKC_v needs to be verified before the communication using SK_v starts. In addition, before generating a session key, DIC_v and GVC must be verified at G and V , respectively. Therefore, DIC_v and GVC help maintain forward secrecy, whose prior verifications are required to establish a secure session in each authentication.

3) *Untraceability*: Since DIC_v is verified based on R_v and N_v , an adversary cannot use DIC_v of V . Due to the freshness of these values in a new mutual authentication process, the same values cannot be reused in successive secure session establishments. Thus, the untraceability of UAVs and authentication parameters are ensured.

4) *Clock Synchronization Avoidance*: Timestamps are not used by SwarmAuth. Therefore, it can avoid clock synchronization issues and the time delay in message transmissions.

5) *Secure Key Agreement*: The secrecy of N_v and R_v is proved in Section VI-A, and these parameters are used to verify

TABLE II
COMPARISON OF BASELINE MECHANISMS

Item	Contributions	UAV-GS Authentication	Mutual Authentication	UAV Swarms	Blockchain	PUFs	Intelligent UAV Cluster Formation
[2]	Proposes a blockchain-enabled distributed authentication solution, which deals with message authentication and key agreement for industrial UAVs.	Yes	No	Yes	Yes	No	No
[6]	Designs an authentication mechanism for UAV swarms, which supports spanning tree-based traversal for multi-hop communications.	Yes	Yes	Yes	No	Yes	No
[12]	Presents a PUF-based authentication protocol for secure communications between base stations and UAVs in UAV swarms.	Yes	Yes	Yes	No	Yes	Yes
[26]	Proposes a PUF-based authentication and attestation scheme for UAV swarms using an optimal trajectory approach.	Yes	Yes	Yes	No	Yes	No
[20]	Proposes a blockchain-based authentication mechanism, which particularly addresses the cross-domain authentication for 5G-enabled UAVs.	Yes	Yes	No	Yes	No	No
SwarmAuth	Proposes a blockchain-based authentication mechanism for UAV swarms, where the GSC and UAVs follow a mutual authentication approach using PUFs, and the K-means clustering scheme is used to dynamically create location-based clusters.	Yes	Yes	Yes	Yes	Yes	Yes

SKC_v at V before using the generated SK_v . Thus, based on SKC_v , the key agreement is securely maintained.

6) *Resistance to Common Attacks*: Since the PUF is hardware-specific, it cannot be accessed by $\mathcal{A}$. Thus, V cannot be masqueraded by $\mathcal{A}$, leading to the failure of masquerade attacks. Due to the encrypted transmission of R_v and N_v , they are good secrets between G and V (proved in Section VI-A), which helps prevent man-in-the-middle attacks. In each authentication approach, a new TID_v is generated, and a new (C_v, R_v) pair, N_v , and DIC_v are used. Consequently, based on old values of these parameters, messages cannot be decoded and replay attacks are prevented. If a device is captured by $\mathcal{A}$, the aforementioned parameters cannot be accessed because they are not stored in V 's or G 's databases. Moreover, since PUFs are inherently unclonable, $\mathcal{A}$ cannot clone V . Therefore, SwarmAuth is secure against node tampering and cloning attacks.

7) *User Anonymity*: TID_v and DIC_v are updated in each new authentication procedure, and the update is based on C_v , N_v , and R_v . Thus, SwarmAuth guarantees user anonymity for every fresh authentication.

VII. PERFORMANCE EVALUATION

In this section, we present a comparative analysis of SwarmAuth and related schemes, such as the mechanisms proposed by Tan et al. [2], Bansal et al. [12], Gaurang et al. [26], Feng et al. [20], and Bansal et al. [6]. In this context, we consider the

computation and communication overheads, along with different security features.

A. Baseline Mechanisms

The authors in [2] design a blockchain-enabled authentication mechanism for industrial UAVs. The blockchain peer nodes maintain a ledger that stores authentication information, and using APIs, UAVs execute smart contracts to facilitate the authentication process, which applies elliptic curve cryptography. Considering a group of UAVs, the authors in [12] propose an authentication protocol for the communication between base stations and UAVs. Based on the UAVs' locations, UAV clusters are formed using the K-Means clustering scheme, which helps reduce the message propagation time. In [20], a blockchain-based cross-domain authentication mechanism is proposed for 5G-enabled drones, where the identity of UAVs is dynamically managed using a multi-signature-based smart contract. In that work, the authors design a digital identity credential for UAVs. The work in [26] proposes a PUF-based mutual authentication and attestation protocol for UAV swarms with base stations. In that work, based on an optimal messaging path, a Christofides algorithm is used to find an optimal trajectory to attain scalability. In [6], the authors design a PUF-based mutual authentication protocol for UAV swarm networks, where a spanning tree-based traversal is used to support multi-hop communication and dynamic topologies considering mobile UAVs. Table II presents

TABLE III
SECURITY FEATURES COMPARISON

Feature	Tan <i>et al.</i> [2]	Bansal <i>et al.</i> [12]	Gaurang <i>et al.</i> [26]	Feng <i>et al.</i> [20]	Bansal <i>et al.</i> [6]	SwarmAuth
Mutual authentication	✓	✗	✓	✓	✓	✓
Secure key agreement	✓	✗	✗	✗	✗	✓
Avoidance of clock synchronization	✗	✗	✗	✗	✗	✓
Confidentiality	✓	✗	✓	✗	✓	✓
Forward secrecy	✓	✗	✗	✗	✗	✓
User anonymity	✗	✗	✗	✗	✗	✓
Man-in-the-middle attacks	✓	✓	✓	✓	✓	✓
Masquerade attacks	✓	✓	✓	✓	✓	✓
Replay attacks	✓	✓	✓	✓	✓	✓
Node tampering attacks	✓	✓	✓	✗	✓	✓
Cloning attacks	✓	✓	✓	✗	✓	✓
Untraceability property	✗	✗	✗	✗	✗	✓

a comparison of baseline mechanisms with our approach. In the proposed blockchain-based authentication mechanism, both UAV registration and authentication are performed using the blockchain, which stores the information related to the UAV registration. Therefore, both the GSC and the UAV can use untampered information for their mutual authentication.

B. Security Features Comparison

The security feature comparison is presented in Table III, where '✓' represents the satisfaction of a criterion, and '✗' denotes that the corresponding criterion is not fulfilled by the mechanism. From Table III, it is noted that SwarmAuth provides superior security features than baselines. SwarmAuth and baselines protect against common attacks, such as man-in-the-middle, masquerade, replay, and node tampering attacks. The mutual authentication is not satisfied by the scheme in [12]. The secure key agreement and forward secrecy are satisfied by SwarmAuth and [2]. Apart from [20], other baselines and SwarmAuth use PUFs and thus can guard against node tampering and cloning attacks. The clock synchronization avoidance feature, user anonymity, and untraceability property are failed by all the baselines.

C. Computation Cost Comparison

Let T_a , T_e , and T_h define the notations of computing analog extractor operations, encryption/decryption, and hash operations, respectively. The costs of T_a , T_e , and T_h are taken from the works in [37], [38], [39]. Considering the experimental results given in [37], [38], [39], we have $T_a \approx 2.045$ ms, $T_e \approx 8.7$ ms, and $T_h \approx 0.5$ ms.

Considering the authentication process, Table IV highlights the computational cost of SwarmAuth and baselines. In that table, it is noted that SwarmAuth has a lower cost than other baselines. In SwarmAuth, G and V need to perform $4T_h + T_e$ and $5T_h + T_e$ operations, respectively, which yields a total cost of $9T_h + 2T_e$ operations, i.e., 21.9 ms. In this context, T_e is associated with the authentication for the group message transmission. For [2], the numbers of hash (T_h) and encryption/decryption (T_e) operations are 4 and 3, respectively, which are primarily associated with the authentication and key agreement, and thus the computation cost is 38.3. The mechanisms in [12] and [26] do not use hash functions and their computation cost is 34.8 ms, which primarily depends on the encryption/decryption. In [20],

TABLE IV
COMPUTATION COST COMPARISON

Scheme	UAV	GSC	Total cost
SwarmAuth	$5T_h + T_e$ ≈ 11.2	$4T_h + T_e$ ≈ 10.7	$9T_h + 2T_e$ ≈ 21.9 ms
Tan <i>et al.</i> [2]	$7T_h + 4T_e$ ≈ 38.3	–	$7T_h + 4T_e$ ≈ 38.3 ms
Bansal <i>et al.</i> [12]	$2T_e$ ≈ 17.4	$2T_e$ ≈ 17.4	$4T_e$ ≈ 34.8 ms
Gaurang <i>et al.</i> [26]	$2T_e$ ≈ 17.4	$2T_e$ ≈ 17.4	$4T_e$ ≈ 34.8 ms
Feng <i>et al.</i> [20]	$2T_h + 10T_e$ ≈ 88	–	$2T_h + 10T_e$ ≈ 88 ms
Bansal <i>et al.</i> [6]	$2T_e$ ≈ 17.4	$T_h + 2T_e$ ≈ 17.9	$T_h + 4T_e$ ≈ 35.3 ms

TABLE V
COMMUNICATION COST COMPARISON

Scheme	No. of messages	No. of bits
SwarmAuth	2	480
Tan <i>et al.</i> [2]	3	1088
Bansal <i>et al.</i> [12]	2	992
Gaurang <i>et al.</i> [26]	2	576
Feng <i>et al.</i> [20]	6	1056
Bansal <i>et al.</i> [6]	2	416

the sum of session key negotiation and intra-domain authentication cost is 88 ms. Considering the authentication and attestation for a UAV, the computation cost in [6] is based on hash and encryption/decryption operations.

D. Communication Cost Comparison

In order to calculate the communication cost in the UAV authentication, we consider that the hash output, identity, random nonce, and each of the PUF challenge and response contain 160, 160, 128, and 128 bits, respectively. To compute the hash, we apply secure hash algorithm (SHA-1) [40]. For baselines, the elliptic curve cryptography (ECC) point, ciphertext block (using the AES-128), and timestamp contain 320, 128, and 32 bits, respectively. Table V represents the communication cost of our and baseline mechanisms.

In SwarmAuth, during the authentication of a UAV, the total number of communicated messages is 2. The first $\langle TID_v \rangle$ and the second $\langle GVC, SKC_v \rangle$ messages require 160 and $160 + 160 = 320$ bits, respectively, and thus our approach has a total

TABLE VI
TIME COST OF APIs IN THE SMART CONTRACT

Smart contract APIs	Average time (s)		
	4 blockchain nodes	6 blockchain nodes	8 blockchain nodes
<i>saveSI</i>	1.2	1.8	2.1
<i>deleteSI</i>	1.14	1.62	2.12
<i>updateSI</i>	1.1	1.6	1.9
<i>queryPQ</i>	0.664	0.667	0.669
<i>queryDIC</i>	0.684	0.686	0.687

communication cost of 480 bits. From Table V, it is noted that the mechanism in [2] has the highest communication cost. Although the scheme in [6] has the lowest communication overhead, it does not use any hash operation, which is useful to verify the correct reception of a message. The mechanism proposed in [20] requires the largest number of message communications.

E. Evaluation of Blockchain on Ethereum Virtual Machine

We implement the proposed blockchain in Truffle, which is a popular blockchain development and testing framework using the Ethereum Virtual Machine (EVM). The blockchain peer nodes are hosted on the cloud server. First, we compute the time cost of the smart contract APIs, which are designed in SwarmAuth. Second, we test the permission scheme of our blockchain-based authentication service, which is crucial because it can directly impact the security feature of the system. Each API is executed 50 times, and Table VI shows the results of the APIs' computing time. For the consensus algorithm, kafka is used. To meet the endorsement policy, each transaction proposal needs to be endorsed by a minimum of 50% of the blockchain nodes. Since *saveSI*, *deleteSI*, and *updateSI* APIs involve modifications of the SIL, they are recorded in transactions, which are packaged into blocks, then all UAVs are informed about the modification of the SIL. However, *queryPQ* and *queryDIC* APIs only query the SIL and do not create any new block. As a result, the average time of *saveSI*, *deleteSI*, and *updateSI* is higher than *queryPQ* and *queryDIC*, as shown in Table VI. In SwarmAuth, in order to prepare the next authentication session, a UAV can call the APIs to update its own record after each current authentication session. Thus, during the authentication phase, it is not required for UAVs to wait for the results of the smart contract execution.

The performance of the blockchain network depends on the time cost of the smart contract APIs. If the number of blockchain nodes increases, the number of traffic to access the blockchain nodes also increases. Consequently, the number of executions of the smart contract APIs increases, which can impact the average time cost of the APIs and thus the performance of the blockchain. In order to evaluate the performance of the blockchain network, we have computed the time cost of the smart contract APIs with different number of blockchain peer nodes, and the results are shown in Table VI. From this table, it is noted that, as the number of peer nodes in the blockchain increases, the time costs of *saveSI*, *deleteSI*, and *updateSI* increase compared to

queryPQ and *queryDIC*. This is because *saveSI*, *deleteSI*, and *updateSI* APIs modify SIL, which involves recording transactions, packaging into blocks, and informing all UAVs about the update of SIL. Whereas, *queryPQ* and *queryDIC* APIs only query SIL and do not generate any new blocks. Therefore, if the number of UAVs increases, the number of blockchain nodes can be increased. However, as a result, the average time cost of the smart contract APIs will be higher.

In case of the permission scheme evaluation, only the GSC can add a new UAV or delete an existing UAV to/from the blockchain. Fig. 9(a) shows the scenarios where the GSC successfully adds a new UAV UAV1. The *validate_id* variable is first used to validate the user. When the value of *validate_id* is "GSC", *validate_id* represents the GSC. Thus, the details of UAV1 can be successfully added. In Fig. 9(b), the value of *validate_id* is "UAV1", i.e., a UAV, which sends a proposal to add another UAV UAV2. Thus, the proposal fails to execute and UAV1 gets an error message. A UAV can only update its own information in the SIL, as illustrated in Fig. 10(a), where UAV1 modifies its record. Since a UAV does not have the permission to update the record of another UAV, UAV1 cannot update UAV2's information, as shown in Fig. 10(b). In this context, UAV1 sends a transaction proposal to blockchain nodes to modify UAV2's record; however, the permission is denied.

F. Simulation Scenario

We implement SwarmAuth in NS-3.36 [31] with a 5 GHz IEEE 802.11ac network. We set the area of the simulation to a $500 \times 500 \times 300\text{m}$ region, with one GSC and several swarms of UAVs, where the GSC is placed at the center of the region. In particular, n is the total number of UAVs in the network. In the simulation, we consider 500 UAVs. In the simulation, we consider 500 UAVs, and we create multiple clusters dynamically, where each cluster has a maximum number of 10 UAVs. In the performance analysis of SwarmAuth, we show the results of a single cluster among the multiple clusters formed in the simulation. The results are the average performances of the UAVs in a cluster. The UAVs fly within $\{[0, 500], [0, 500], [0, 300]\}$ at a maximum speed of 50 m/s. The analysis of the throughput performance is based on the UDP throughput. The duration of each simulation experiment is 100s, and each experiment is run 10 times. Thus, results are computed as an average of 10 runs. Table VII presents the simulation parameters with their values.

G. Communication and Computation Cost Analysis Under Different Cluster Size

From Tables IV and V, the communication and computation costs are reduced in SwarmAuth; however, the costs increase as the cluster size increases, as shown in Fig. 11. When the number of UAVs is 10, SwarmAuth has lower overheads than the schemes proposed in [2], [12], [26], and [20]. Specifically, SwarmAuth has approximately 38%, 37%, 27%, and 3.70% less communication cost than the schemes proposed in [2], [12], [20], and [26], respectively. As given in Table IV, SwarmAuth has marginally higher communication overhead than that of Bansal



Fig. 9. (a) GSC successfully adds UAV1 (b) UAV1 does not have permission to add UAV2.



Fig. 10. (a) UAV1 can update its own record (b) UAV1 cannot update UAV2's record.

TABLE VII
SIMULATION PARAMETERS

Parameter	Value
Area	$500m \times 500m \times 300m$
Wireless standard	IEEE 802.11ac
Channel bandwidth	40 MHz
Data and control mode	Constant rate wifi manager
Maximum physical data rate	41 Mbps
Path loss model	Log-normal path loss model (path loss exponent=3.0)
Propagation delay model	Constant speed propagation delay model
Mobility model	Random walk 2d mobility model ("Mode: Time", "Time: 2s", "Bounds: Rectangle (0, 500, 0, 500)", "Speed: UniformRandomVariable[Min=2.0, Max=4.0]", "Direction: UniformRandomVariable[Min=0.0, Max=6.283184]")
UAVs' maximum transmit power	5 W
Noise power	-110 dBm
Noise power spectral	-170 dBm/Hz

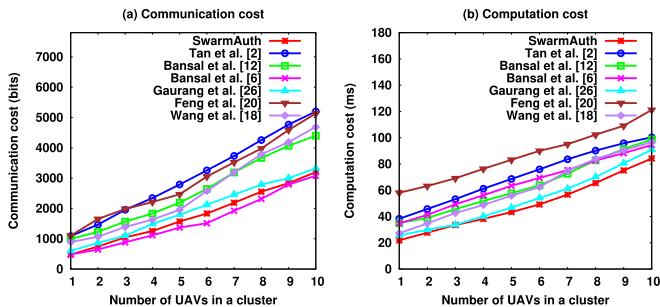


Fig. 11. Impact of cluster size on: (a) Communication cost and (b) Computation cost.

et al. [6]. As a result, the communication cost in SwarmAuth is approximately 4% higher than that of the work in [6].

In our mechanism, the computation overheads are approximately 31%, 16%, 14%, 11%, and 7.27% lower than that of [2], [6], [12], [20], and [26], respectively. This is because the computation overheads of the authentication related operations are higher in baselines than SwarmAuth. Since the work in [18] tries to reduce the computational cost of the authentication approach, we also compare the communication and computation costs of SwarmAuth with the scheme proposed in [18]. From Fig. 11, it is noted that our SwarmAuth has approximately 32% and 13% lower communication and computational overheads than that of [18], respectively. This is because the scheme in [18] runs three authentication related algorithms.

In Fig. 11, the communication and computation costs are computed under different numbers of UAVs in a cluster, where a UAV can join or leave the cluster dynamically. Since we consider a maximum of 10 UAVs in a cluster, the number of UAVs varies from 1 to 10 in Fig. 11. Therefore, this figure captures the impact of dynamically changing the number of UAVs in a cluster on the performance of SwarmAuth.

H. Throughput and Delay Analysis

The throughput analysis is shown in Fig. 12(a). The throughput of SwarmAuth is approximately 23.46%, 8.17%, 21.04%, and 10.52% higher than that of Tan et al. [2], Bansal et al. [12], Gaurang et al. [26], and Feng et al. [20], respectively. This is because the communication cost of SwarmAuth is lower than that of the aforementioned baselines, as shown in Table V. As shown in Fig. 12(b), the delay in SwarmAuth is approximately 8%, 3.85%, 3.22%, and 13.83% lower than the delays in Tan et al. [2], Bansal et al. [12], Gaurang et al. [26], and Feng et al. [20],

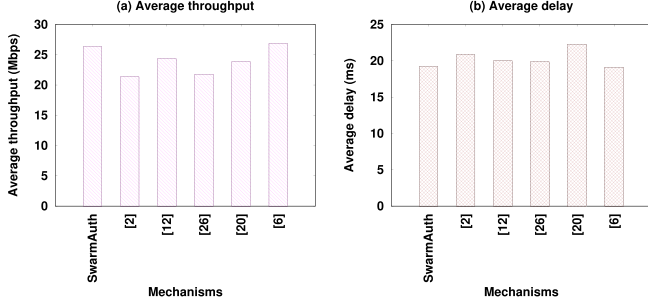


Fig. 12. (a) Average throughput and (b) Average delay.

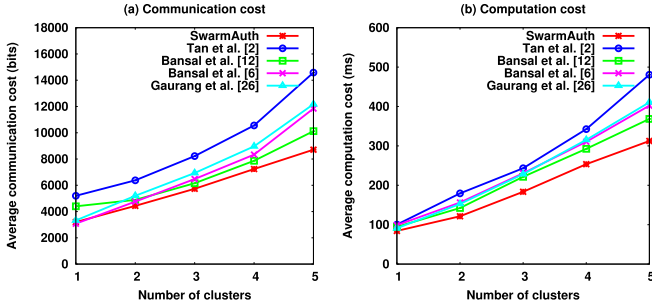


Fig. 13. Impact of number of clusters on: (a) Communication cost and (b) Computation cost.

respectively. Since small sized messages are used in SwarmAuth, the delay is reduced in the proposed mechanism. Since the mechanism proposed in Bansal et al. [6] has marginally lower communication cost than SwarmAuth, the average throughput and delay in [6] are approximately 2% higher and 0.48% lower than that of SwarmAuth, respectively.

I. Impact of Dynamic Cluster Formation on Network Performance

In order to analyze the impact of the dynamic cluster formation on the performance of SwarmAuth, we consider 5 clusters and compute the average communication and computation costs, throughput, and delay of the 5 clusters in the network. Since the works in [2], [6], [12], and [26] consider multiple clusters of UAVs, we include these baselines in the performance analysis of the dynamic cluster formation.

1) *Communication and Computation Cost Analysis Under Different Number of Clusters:* From Fig. 13, it is noted that when multiple clusters are considered to compute the average communication and computation costs in the network, these average costs increase as the number of clusters increases. However, since SwarmAuth uses a K-means clustering-based intelligent approach to dynamically form clusters, our proposed mechanism has lower costs than baselines. For instance, when the number of clusters is 5, SwarmAuth has approximately 14%, 26%, 28%, and 40% lower average communication cost than that of the works in [6], [12], [26], and [2], respectively, as shown in Fig. 13(a). In the case of the computation overhead, in SwarmAuth, the average computation cost is approximately 15%, 22%, 24%, and 34% lower than that of the works in [6],

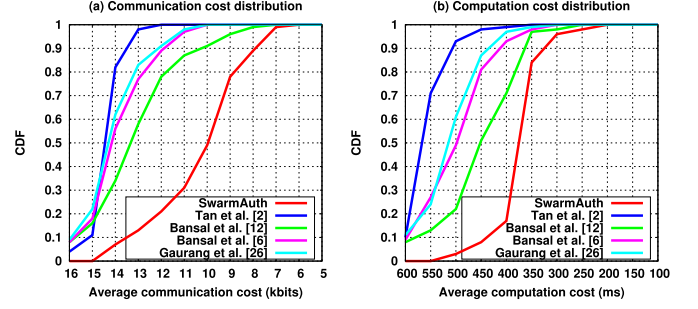


Fig. 14. (a) Communication cost distribution and (b) Computation cost distribution.

[12], [26], and [2], respectively, as shown in Fig. 13(b). The works in [6] and [26] do not use any learning-based approach for the dynamic cluster creation. The work in [2] does not deal with dynamic cluster formation. Although the work in [12] uses a K-means clustering scheme for the cluster creation, the impact of the UAV's movement on the recomputation of the cluster center is not included in the cluster update.

2) *Communication and Computation Cost Distribution:* In order to analyze the impact of the UAVs' high mobility on the stability of the clusters, we run the simulation for 200 continuous iterations considering 5 clusters and calculate the average communication and computation costs in the network after each iteration. Then, we compute the cumulative distribution function (CDF) of the average communication and computation costs, and the results are shown in Fig. 14. Since our simulation scenario (Section VII-F) emulates a practical environment, Fig. 14 captures the performance of SwarmAuth in practical scenarios. In Fig. 14, it is noted that SwarmAuth has higher CDFs of the aforementioned costs than baselines. In SwarmAuth, the cluster centers are updated based on the number of UAVs in a cluster and the positions of the UAVs. That approach helps reconstruct clusters dynamically when there is a change in the number of UAVs in a cluster.

Moreover, when a UAV travels further than a distance of $d_{cv_p}^{min}$, the distance between the UAV and the corresponding cluster center is recomputed. Consequently, when a UAV travels less than or equal to a distance of $d_{cv_p}^{min}$, a cluster is not affected by the frequent movement of the UAV. As a result, a balance is maintained between the stability of a cluster and the high mobility of the UAVs. Thus, SwarmAuth uses a better adaptive cluster formation approach than the work in [12]. Fig. 14(a) shows that, in SwarmAuth, the density of the communication cost spans through 7 – 14 kbits; whereas, in baselines, the distributions are high in the 9 – 15 kbits range. From Fig. 14(b), it is noted that the density of the computation cost of SwarmAuth is high in the 250 – 500 ms range. In that case, baselines have high CDFs in the 350 – 600 ms range.

3) *Throughput and Delay Analysis Under Different Number of Clusters:* Fig. 15 shows the average throughput and delay analysis under different number of clusters in the network. For instance, when the number of clusters is 5 in Fig. 15(a), the average throughput of SwarmAuth is approximately 8.59%, 14.34%, 23.13%, and 66.25% higher than that of Bansal et

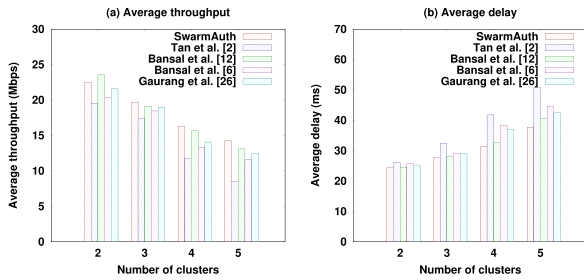


Fig. 15. (a) Average throughput and (b) Average delay.

al. [12], Gaurang et al. [26], Bansal et al. [6], and Tan et al. [2], respectively. This is because, under a higher number of clusters, the average communication cost of SwarmAuth is lower than that of the aforementioned baselines, as shown in Fig. 13(a). Similarly, due to the lower average computation cost in SwarmAuth (shown in Fig. 13(b)), the average delay in SwarmAuth is approximately 6.7%, 11.1%, 16.87%, and 25.8% lower than the delays in Bansal et al. [12], Gaurang et al. [26], Bansal et al. [6], and Tan et al. [2], respectively, as shown in Fig. 15(b).

VIII. CONCLUSION

SwarmAuth uses a blockchain-based distributed mechanism to provide a UAV authentication service for UAV swarms, where the authentication approach imposes a PUF-based mutual authentication between the GSC and a UAV. The ledger maintained by the blockchain peer nodes stores authentication information, and the smart contract APIs control the access to the ledger to perform trustworthy operations facilitating the UAV authentication process. Moreover, considering the locations of UAVs, the K-means clustering scheme helps dynamically create UAV clusters, and consequently the overall network performance is improved. Both formal verification and informal proofs of SwarmAuth are proved considering general attacks. To analyze the practical impact of SwarmAuth, we implement the proposed blockchain in Truffle-based EVM, and SwarmAuth is evaluated through simulations, where the results show the effectiveness of SwarmAuth for UAV swarms and its superior performance compared to baselines. In our future work, we shall add intelligent cross-domain authentication to further improve the security of UAV swarms.

ACKNOWLEDGMENTS

Any opinions and conclusions in this work are strictly those of the author(s) and do not reflect the views, positions, or policies of-and are not endorsed by-IDEAs, DND, or the Government of Canada.

REFERENCES

- [1] H. Shakhatareh et al., "Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges," *IEEE Access*, vol. 7, pp. 48 572–48 634, 2019.
- [2] Y. Tan, J. Wang, J. Liu, and N. Kato, "Blockchain-assisted distributed and lightweight authentication service for industrial unmanned aerial vehicles," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 16928–16940, Sep. 2022.

- [3] Y. Yan, Y. Qian, H. Sharif, and D. Tipper, "A survey on cyber security for smart grid communications," *IEEE Commun. Surveys Tuts.*, vol. 14, no. 4, pp. 998–1010, Fourth Quarter 2012.
- [4] J. Srinivas, A. K. Das, N. Kumar, and J. J. Rodrigues, "TCALAS: Temporal credential-based anonymous lightweight authentication scheme for Internet of drones environment," *IEEE Trans. Veh. Technol.*, vol. 68, no. 7, pp. 6903–6916, Jul. 2019.
- [5] D. Wang, P. Wang, C.-G. Ma, and Z. Chen, "Robust smart card based password authentication scheme against smart card security breach," *Cryptol. ePrint Arch.*, 2012.
- [6] G. Bansal and B. Sikdar, "S-MAPS: Scalable mutual authentication protocol for dynamic UAV swarms," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 12 088–12 100, Nov. 2021.
- [7] M. Gharibi, R. Boutaba, and S. L. Waslander, "Internet of drones," *IEEE Access*, vol. 4, pp. 1148–1162, 2016.
- [8] Y. Sun, J. Liu, K. Yu, M. Alazab, and K. Lin, "PMRSS: Privacy-preserving medical record searching scheme for intelligent diagnosis in IoT healthcare," *IEEE Trans. Ind. Inform.*, vol. 18, no. 3, pp. 1981–1990, Mar. 2022.
- [9] G. Bansal, N. Naren, V. Chamola, B. Sikdar, N. Kumar, and M. Guizani, "Lightweight mutual authentication protocol for V2G using physical unclonable function," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7234–7246, Jul. 2020.
- [10] M. Zhaofeng, W. Lingyun, W. Xiaochang, W. Zhen, and Z. Weizhe, "Blockchain-enabled decentralized trust management and secure usage control of IoT big data," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4000–4015, May 2020.
- [11] T. Jiang, H. Fang, and H. Wang, "Blockchain-based internet of vehicles: Distributed network architecture and performance analysis," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4640–4649, Jun. 2019.
- [12] G. Bansal and B. Sikdar, "Location aware clustering: Scalable authentication protocol for UAV swarms," *IEEE Netw. Lett.*, vol. 3, no. 4, pp. 177–180, Dec. 2021.
- [13] K. Ranjitha, D. Pathak, P. Tammana, A. Franklin A, and T. Al-ladi, "Accelerating PUF-based UAV authentication protocols using programmable switch," in *Proc. 14th Int. Conf. Commun. Syst. Netw.*, 2022, pp. 309–313.
- [14] H. Wang, H. Fang, and X. Wang, "Safeguarding cluster heads in UAV swarm using edge intelligence: Linear discriminant analysis-based cross-layer authentication," *IEEE Open J. Commun. Soc.*, vol. 2, pp. 1298–1309, 2021.
- [15] T. Alladi, Naren, G. Bansal, V. Chamola, and M. Guizani, "SecAuthUAV: A novel authentication scheme for UAV-ground station and UAV-UAV communication," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15 068–15 077, Dec. 2020.
- [16] B. Semal, K. Markantonakis, and R. N. Akram, "A certificateless group authenticated key agreement protocol for secure communication in untrusted UAV networks," in *Proc. IEEE/AIAA 37th Digit. Avionics Syst. Conf.*, 2018, pp. 1–8.
- [17] M. Yahuza, M. Y. I. Idris, A. W. A. Wahab, T. Nandy, I. B. Ahmedy, and R. Ramli, "An edge assisted secure lightweight authentication technique for safe communication on the internet of drones network," *IEEE Access*, vol. 9, pp. 31 420–31 440, 2021.
- [18] H. Wang, H. Fang, and X. Wang, "Edge intelligence enabled soft decentralized authentication in UAV swarm," in *Proc. IEEE/CIC Int. Conf. Commun. China*, 2021, pp. 86–91.
- [19] X. Lan, X. Tang, D. Zhai, D. Wang, and Z. Han, "Blockchain-secured data collection for UAV-Assisted IoT: A DDPG approach," in *Proc. IEEE Glob. Commun. Conf.*, 2021, pp. 1–6.
- [20] C. Feng, B. Liu, Z. Guo, K. Yu, Z. Qin, and K.-K. R. Choo, "Blockchain-based cross-domain authentication for intelligent 5G-enabled internet of drones," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 6224–6238, Apr. 2022.
- [21] G. Li, B. He, Z. Wang, X. Cheng, and J. Chen, "Blockchain-enhanced spatiotemporal data aggregation for UAV-assisted wireless sensor networks," *IEEE Trans. Ind. Inform.*, vol. 18, no. 7, pp. 4520–4530, Jul. 2022.
- [22] A. Aftab, N. Ashraf, H. K. Qureshi, S. A. Hassan, and S. Jangsher, "BLOCK-ML: Blockchain and machine learning for UAV-BSSs deployment," in *Proc. IEEE 92nd Veh. Technol. Conf.*, 2020, pp. 1–5.
- [23] Y. Wang, Z. Su, Q. Xu, R. Li, and T. H. Luan, "Lifesaving with RescueChain: Energy-efficient and partition-tolerant blockchain based secure information sharing for UAV-aided disaster rescue," in *Proc. IEEE INFO-COM*, 2021, pp. 1–10.
- [24] I. J. Jensen, D. F. Selvaraj, and P. Ranganathan, "Blockchain technology for networked swarms of unmanned aerial vehicles (UAVs)," in *Proc. IEEE 20th Int. Symp. "A World Wireless Mobile Multimedia Netw."*, 2019, pp. 1–7.

- [25] S. Sahoo, A. K. Shukla, and H. Rana, "Future directions of UAV swarm communications," in *Proc. 8th Int. Conf. Signal Process. Integr. Netw.*, 2021, pp. 324–327.
- [26] G. Bansal, N. Naren, V. Chamola, and B. Sikdar, "SHOTS: Scalable secure hardware based authentication-attestation protocol using optimal trajectory in UAV swarms," *IEEE Trans. Veh. Technol.*, vol. 71, no. 6, pp. 5827–5836, Jun. 2022.
- [27] K. Gai, Y. Wu, L. Zhu, K.-K. R. Choo, and B. Xiao, "Blockchain-enabled trustworthy group communications in UAV networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4118–4130, Jul. 2021.
- [28] G. Raja, S. Anbalagan, A. Ganapathisubramanian, M. S. Selvakumar, A. K. Bashir, and S. Mumtaz, "Efficient and secured swarm pattern multi-UAV communication," *IEEE Trans. Veh. Technol.*, vol. 70, no. 7, pp. 7050–7058, Jul. 2021.
- [29] W. Mao and C. Boyd, "Towards formal analysis of security protocols," in *Proc. Comput. Secur. Found. Workshop VI*, 1993, pp. 147–158.
- [30] The scyther tool, 2016. Accessed: May 10, 2022. [Online]. Available: <https://people.cispa.io/cas.cremers/scyther/>
- [31] Ns-3.36 - nsnam. [Online]. Available: <https://www.nsnam.org/releases/ns-3--36>
- [32] P. Gupta and P. R. Kumar, "The capacity of wireless networks," *IEEE Trans. Inf. Theory*, vol. 46, no. 2, pp. 388–404, Mar. 2000.
- [33] K. Xu, X. Hong, and M. Gerla, "An ad hoc network with mobile backbones," in *Proc. IEEE Int. Conf. Commun.*, 2002, pp. 3138–3143.
- [34] K. Kandali, L. Bennis, and H. Bennis, "A new hybrid routing protocol using a modified K-means clustering algorithm and continuous hopfield network for VANET," *IEEE Access*, vol. 9, pp. 47 169–47 183, 2021.
- [35] D. Wang, D. He, P. Wang, and C.-H. Chu, "Anonymous two-factor authentication in distributed systems: Certain goals are beyond attainment," *IEEE Trans. Dependable Secure Comput.*, vol. 12, no. 4, pp. 428–442, Jul./Aug. 2015.
- [36] D. Wang, H. Cheng, D. He, and P. Wang, "On the challenges in designing identity-based privacy-preserving authentication schemes for mobile devices," *IEEE Syst. J.*, vol. 12, no. 1, pp. 916–925, Mar. 2018.
- [37] D. He, N. Kumar, M. K. Khan, and J.-H. Lee, "Anonymous two-factor authentication for consumer roaming service in global mobility networks," *IEEE Trans. Consum. Electron.*, vol. 59, no. 4, pp. 811–817, Nov. 2013.
- [38] Q. Jiang, J. Ma, G. Li, and L. Yang, "An efficient ticket based authentication protocol with unlinkability for wireless access networks," *Wireless Pers. Commun.*, vol. 77, no. 2, pp. 1489–1506, 2014.
- [39] Y. Lei, L. Zeng, Y.-X. Li, M.-X. Wang, and H. Qin, "A lightweight authentication protocol for UAV networks based on security and computational resource optimization," *IEEE Access*, vol. 9, pp. 53 769–53 785, 2021.
- [40] S. H. Standard, "FIPS Pub 180-1," *Nat. Inst. Standards Technol.*, vol. 17, no. 180, p. 15, 1995.



Raja Karmakar (Member, IEEE) received the bachelor of technology (BTech) degree in computer science and engineering from the Government College of Engineering and Leather Technology, Kolkata, India, the master of engineering (ME) degree in software engineering from Jadavpur University, Kolkata, India, and the doctor of philosophy (PhD) degree from Jadavpur University, Kolkata, India. Currently, he is an associate professor with the Department of Computer Science and Engineering, Heritage Institute of Technology, Kolkata, India. Prior to that, he was a postdoctoral research fellow with the École de Technologie Supérieure (ÉTS), Université du Québec, Montréal, Canada. His research area includes computer systems, wireless networks, mobile computing, IoT, machine learning, and UAV communications.



Georges Kaddoum (Senior Member, IEEE) received the bachelor's degree in electrical engineering from the École Nationale Supérieure de Techniques Avancées (ENSTA Bretagne), Brest, France, the MS degree in telecommunications and signal processing (circuits, systems, and signal processing) from the Université de Bretagne Occidentale and Telecom Bretagne (ENSTB), Brest, in 2005, and the PhD (honors) degree in signal processing and telecommunications from the National Institute of Applied Sciences (INSA), University of Toulouse, Toulouse, France, in 2009. He is currently an associate professor and Tier 2 Canada research chair with the École de Technologie Supérieure (ÉTS), Université du Québec, Montréal, Canada and associated with Cyber Security Systems and Applied AI Research Center, Lebanese American University, Beirut, Lebanon. In 2014, he was awarded the ÉTS research chair in physical-layer security for wireless networks. Since 2010, he has been a scientific consultant in the field of Space and Wireless Telecommunications for several US and Canadian companies. He has published more than 200+ journal and conference papers and has two pending patents. His recent research activities cover mobile communication systems, modulations, security, and space communications and navigation. He received the Best Papers Awards at the 2014 IEEE International Conference on Wireless and Mobile Computing, Networking, Communications (WIMOB), with three coauthors, and at the 2017 IEEE International Symposium on Personal Indoor and Mobile Radio Communications (PIMRC), with four coauthors. Moreover, he received IEEE Transactions on Communications Exemplary Reviewer Award for the year 2015, 2017, 2019. In addition, he received the research excellence award of the Université du Québec in the year 2018. In the year 2019, he received the research excellence award from the ÉTS in recognition of his outstanding research outcomes. He is currently serving as an associate editor of *IEEE Transactions on Information Forensics and Security*, and *IEEE Communications Letters*.



Ouassima Akhrif (Senior Member, IEEE) received the MSc and PhD degrees in electrical engineering, control systems with the University of Maryland, College Park, USA, in 1987 and 1989, respectively, as a fulbright scholar. After one year as an assistant professor, with the Systems Engineering Department, Case Western Reserve University in Cleveland, she moved to Montreal, Canada, and joined in 1992 the "École de Technologie Supérieure (ÉTS)", Université du Québec, Montréal, Canada, where she is currently a full professor with the Electrical Engineering Department. She is a member of GREPCI (Groupe de Recherche en Commande Industrielle et Électronique de Puissance), a research group that she directed from 2004 to 2009. Her research interests include bifurcation analysis, nonlinear geometric control, nonlinear adaptive control and their applications in electric drives, power systems, renewable energy integration, autopilot design, and flight control systems.